EEE 6778. Applied Machine Learning II


# Module 5. Attention and NLP

## Attention

## Natural Language Processing

# Attention

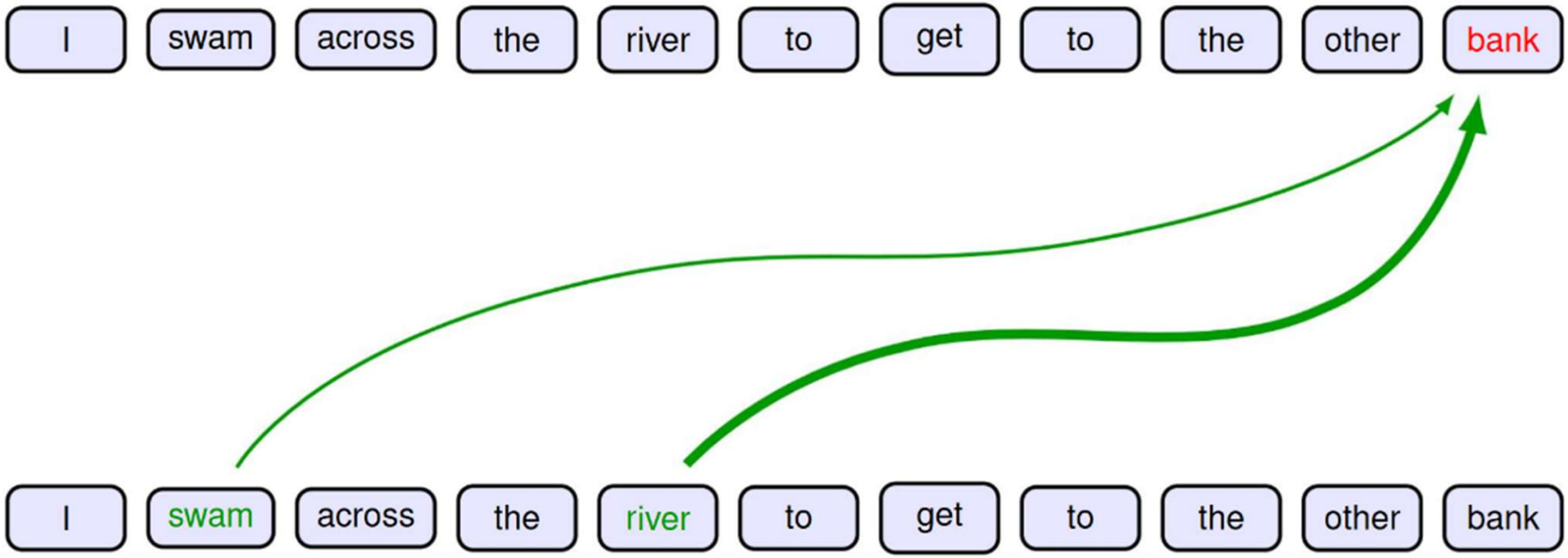
# Attention is all you need

Consider the following two sentences:

- I swam across the river to get to the other **bank**
- I walked across the road to get cash from the **bank**

Bank has different meanings in the two sentences → We need to look at the **context**

# Attention is all you need




Our neural network should **attend** to the other words to rely more heavily on specific ones that give more context

(Bishop, 2024)

# Attention is all you need

If our Query (Q) is “I swam across the river to get to the other **bank**”

And we want to know the context of the word **bank** in Q, we need to understand how the other words (Keys) are similar to Q

**In other words, we need to compute an attention mask:**

How similar is each Key ($K_1$, $K_2$, $K_3$, $K_n$) to the desired Q → **To extract the values with highest attention**

# Self- Attention

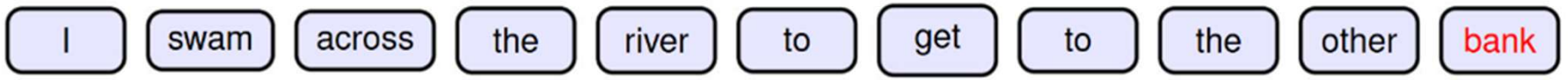
Goal: Identify and attend to most important features in input X


1. Encode **position** information

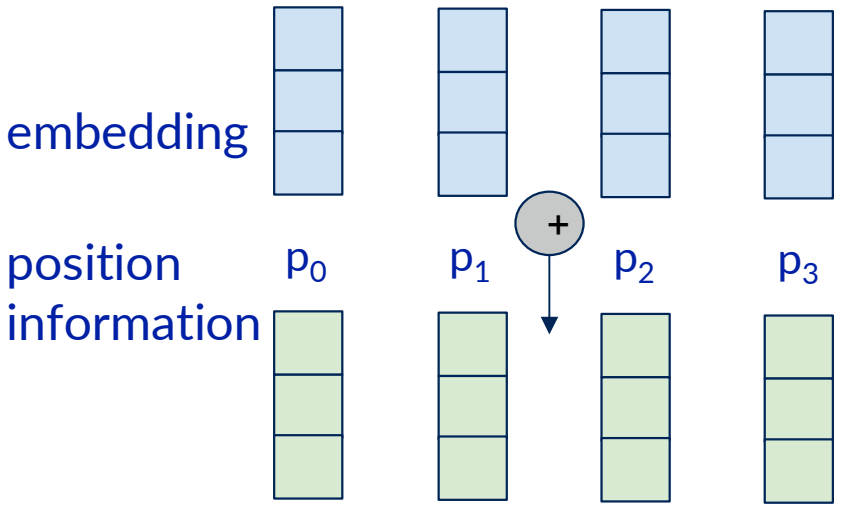
Data is fed at all once. So, we need to encode position information to understand order

# Self- Attention

Goal: Identify and attend to most important features in input X




1. Encode **position** information



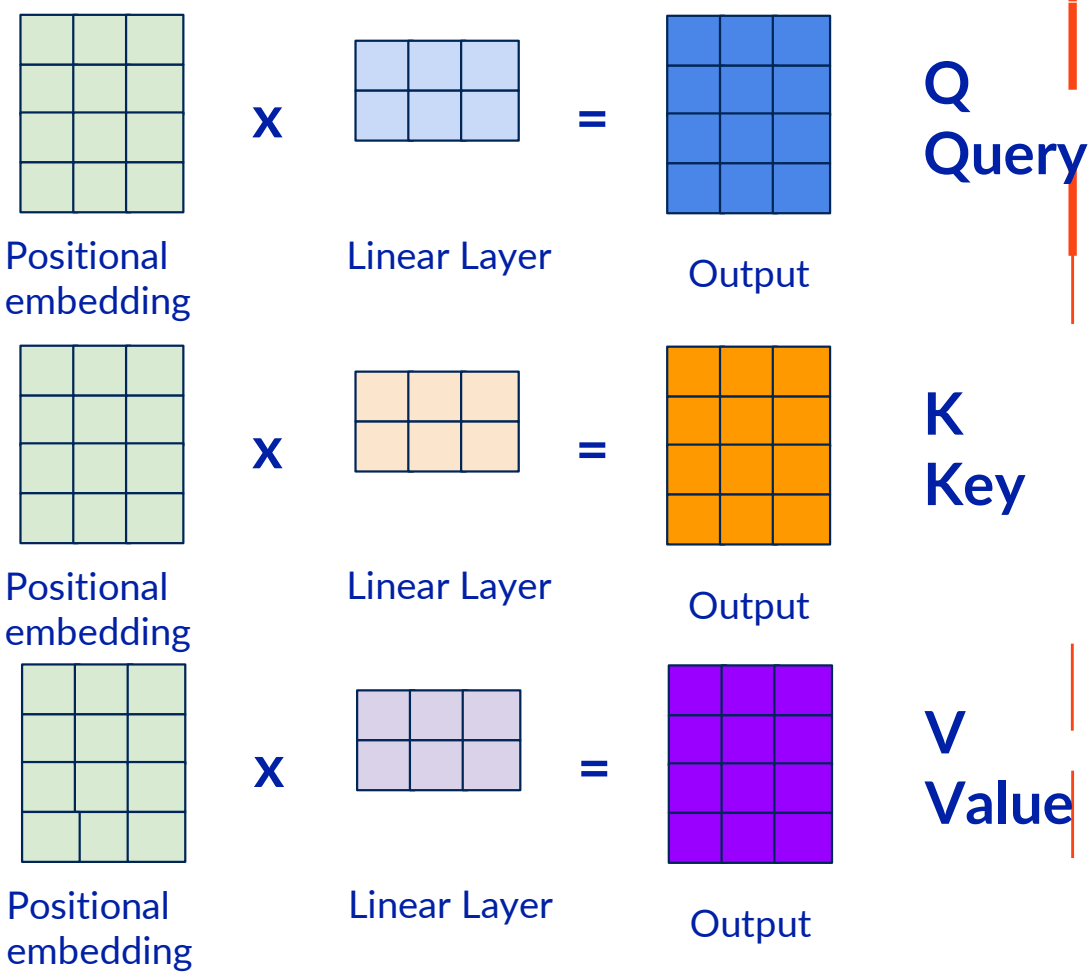

Position-aware encoding

Data is fed at all once. So, we need to encode position information to understand order

# Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search



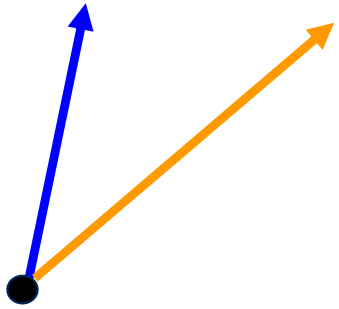
# Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search
3. Compute **attention weighting**

**Attention score**: computer pairwise similarity between each query and key

**How to compute similarity between two sets of features?**



Dot product scale

$$\frac{Q \cdot K^T}{\text{scaling}}$$

Similarity metric

Also known as the "cosine similarity"

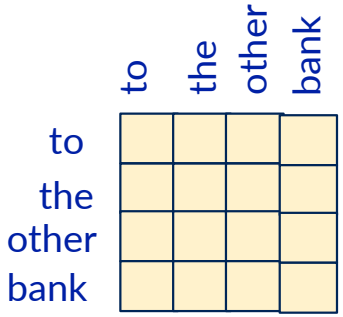
# Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search
3. Compute **attention weighting**

**Attention weighting**: where to attend to

**How similar is the key to the query?**



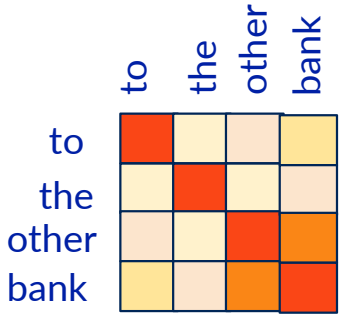
# Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search
3. Compute **attention weighting**

**Attention weighting**: where to attend to

**How similar is the key to the query?**




$$\text{softmax}\left(\frac{Q\ K^T}{\text{scaling}}\right)$$

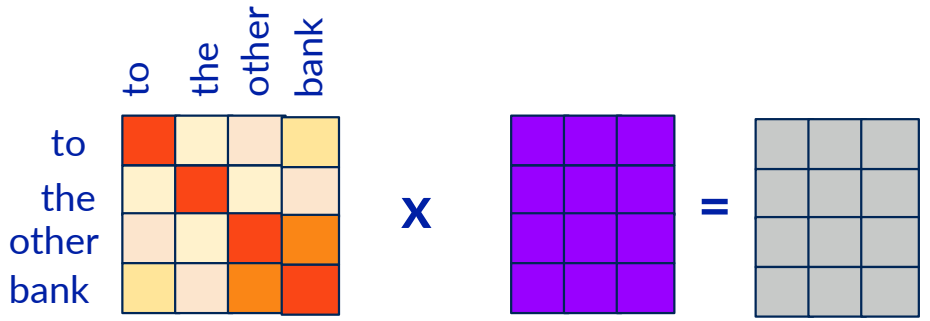
**Attention weighting**

# Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search
3. Compute **attention weighting**
4. Extract features with high attention

**Last step:**
Self-attend to extract features



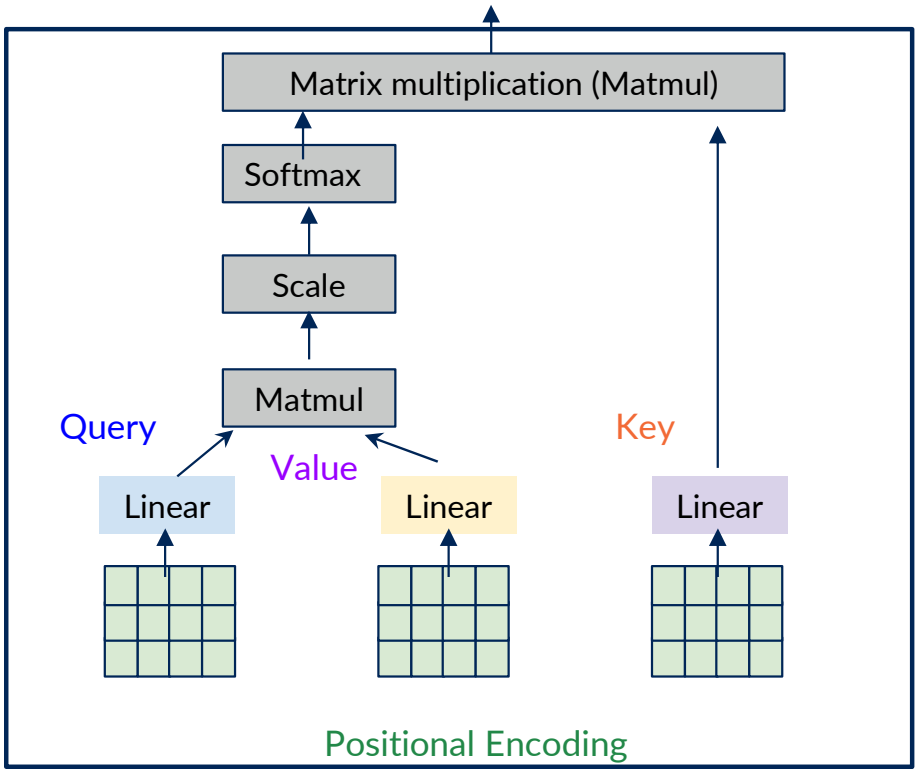

**Attention weighting** **Value (V)** **Output**

$$softmax\left(\frac{Q\ K^T}{scaling}\right) \cdot V \quad = \quad A(Q,K,V)$$

# Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**



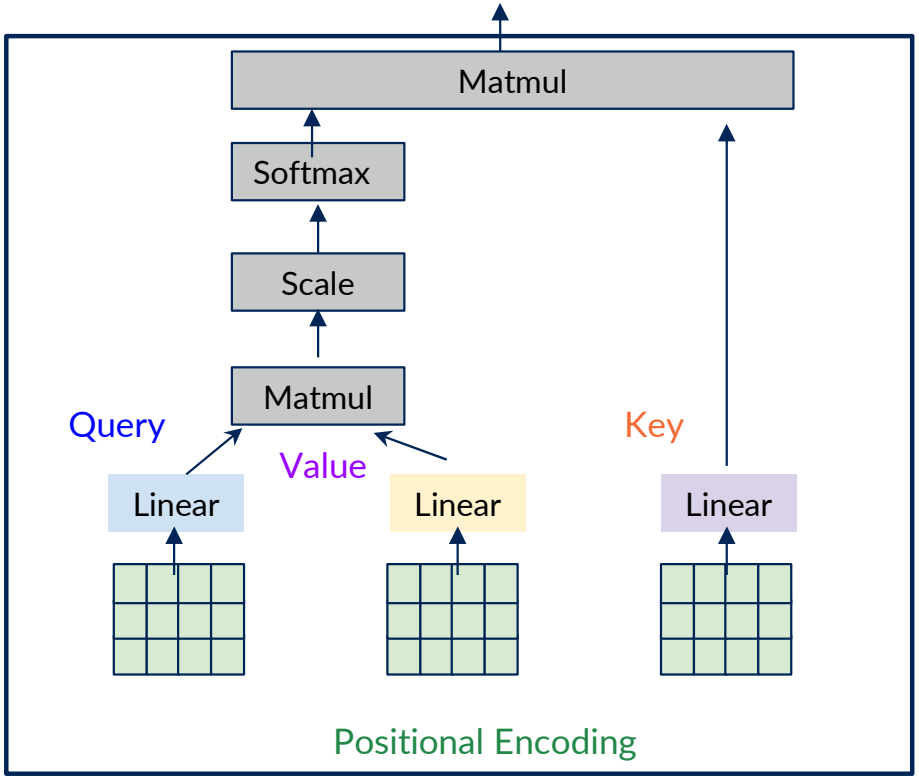

$$\text{softmax}\left(\frac{QK^T}{\text{scaling}}\right)V$$

These operations form a self-attention head that can plug into a larger network. Each head attends to a different part of input.

# Self- Attention

Goal: Identify and attend to most important features in input X

1. Encode **position** information
2. Extract **query, key and value** for search
3. Compute **attention weighting**
4. Extract **features with high attention**




**Attention is the foundational building block of the Transformer architecture**

# Self- Attention - Applications

- Language Processing
  - Natural Language Processing
  - Large Language Models (BERT, GPT)
- Vision Transformers

Next!

# Natural Language Processing

# Introduction

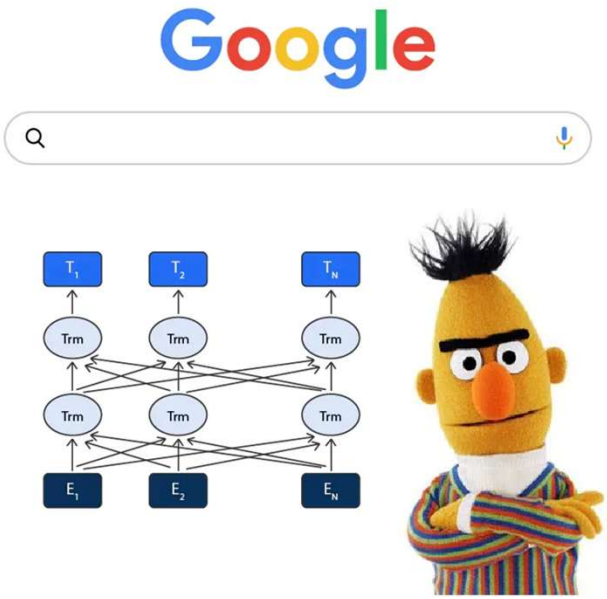
- NLP combines computational linguistics, machine learning, and deep learning models to **process human language.**
- **Language as Sequential Data:** Words in English (and many other languages) form sequences with spaces and punctuation.


## History!

- 2014-2015: LSTMs and GRUs became dominant for sequence tasks like language modeling.
- 2017: Attention Is All You Need (Vaswani et al.) introduced Transformers, replacing RNNs and LSTMs in many NLP tasks.
- 2018: BERT (Google) revolutionized NLP with pretraining and fine-tuning.

# Introduction

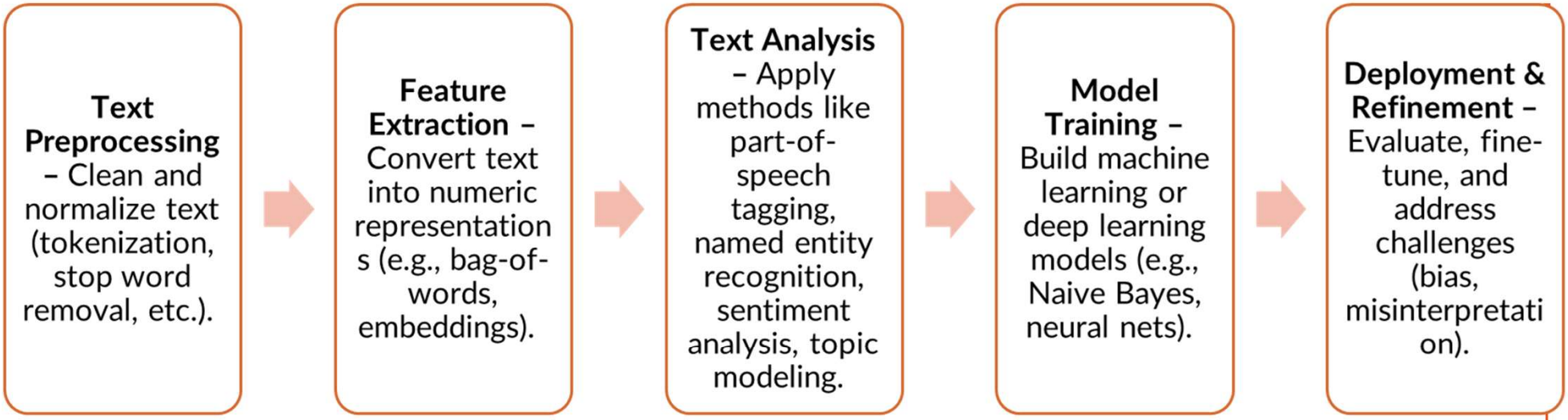
- **BERT (Bidirectional Encoder Representations from Transformers)** was created by Google AI in 2018 to improve contextual understanding in NLP tasks.
- Unlike previous models that processed text **left -to- right(like LSTMs)**, BERT introduced bidirectional context, meaning it learns from both directions simultaneously.



# Why is Language So Complex?

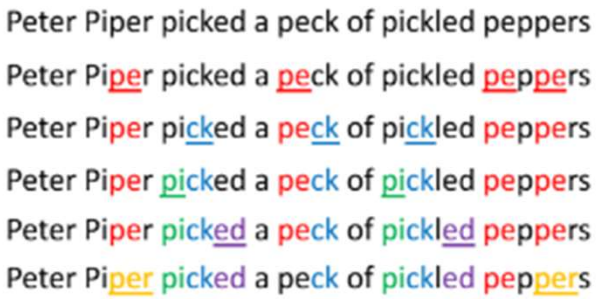
- **Human Language:**
  - Filled with ambiguities, slang, regional dialects.
  - Evolves over time (new words, shifting grammar).
- **Core Linguistic Levels:**
  - **Syntax** – How words form sentences.
  - **Semantics** – Meaning of words/sentences.
  - **Pragmatics** – Context and speaker intent.
  - **Discourse** – Connections across sentences or conversations.

# NLP Pipeline Overview



# Text Preprocessing

- **Tokenization:** Split text into words or subwords.
  - **Byte pair encoding**




- **Lowercasing:** Standardize “Apple” and “apple” as the same token.
- **Stop Word Removal:** Filter out very common words (e.g., “the,” “is,” “and”).
- **Stemming & Lemmatization:** Reduce words to a root form (e.g., “running” → “run”).
- **Text Cleaning:** Remove punctuation, special characters, or numbers if they clutter analysis

# Feature Extraction!! One-Hot

- **One-Hot Vectors**
  - A word $w$ is represented by a vector $x_n$ of length $K$ (vocabulary size).
  - All entries are 0 except for a 1 at the index corresponding to the word.

$$x_n = [\, 0 \,, 0 \,, \dots \,, 1 \,, \dots \,, 0 \,]^T$$

  - **Issue**: Very high dimensional (large $K$) and no sense of similarity (all equidistant).

# Feature Extraction!! Bag-of-words

- Bag-of-Words (BoW): Ignores word order, focuses on frequency of each word.
- **Joint Distribution (assuming independence)**

$$p(x_1, x_2, \ldots, x_N) \approx \prod_{n=1}^{N} p(x_n).$$

  - Here, $x_n$ can be the $n$-th word in the text, drawn from a dictionary of possible words/tokens.
- **Naive Bayes Classification**

$$p(C_k \mid x_1, \ldots, x_N) \propto p(C_k) \prod_{n=1}^{N} p(x_n \mid C_k),$$

  - where $C_k$ is the class (e.g., "positive" vs. "negative" sentiment).

**Pros**: Simple, fast, often effective on short texts (spam detection, quick sentiment tasks).
**Cons**: Loses word order, can't capture context well.

# Feature Extraction!! Word Embeddings

- **Embedding Matrix**
  - We learn a matrix $E$ of size $D \times K$ ( $D$ = embedding dimension and $K$ the dimensionality of the dictionary).
  - The embedding vector for word $x_n$ is

$$v_n = E\, x_n$$

  - Essentially, selects the column of $E$ that corresponds to the word, yielding a dense vector of length $D$.
- **Benefit of Embeddings**
  - Captures semantic relationships; words appearing in similar contexts have closer vectors →The embedding matrix translates words into numbers in a way that preserves meaning.
  - Enables vector arithmetic analogies (e.g., $v$ ( Paris ) − $v$ ( France ) + $v$ ( Italy ) ≈ $v$ ( Rome )

# Feature Extraction!! Word Embeddings

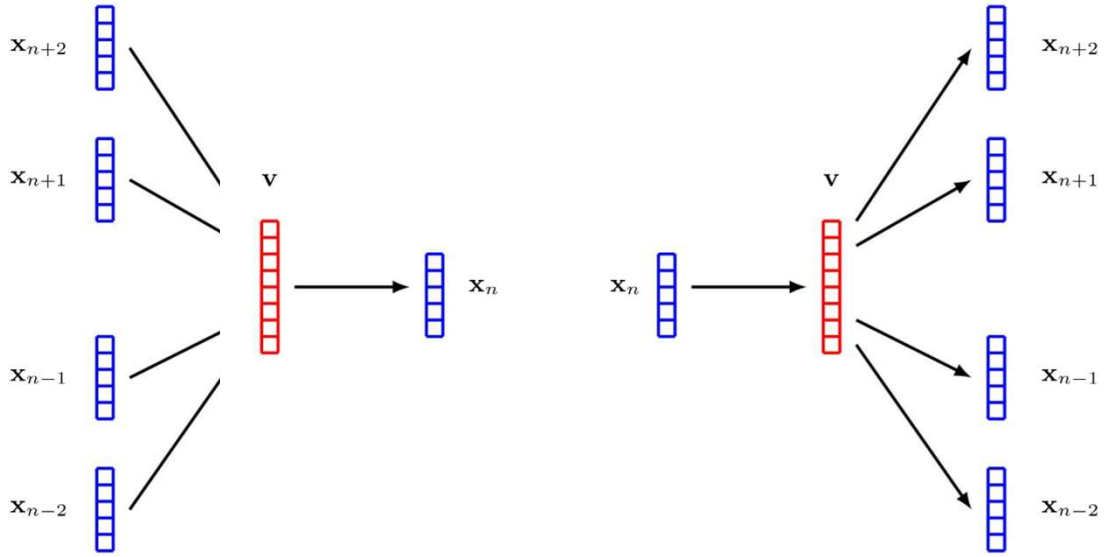
| | apple | banana | cat | dog | elephant |
|---|---|---|---|---|---|
| Dim_1 | 0.3745401188473625 | 0.9507143064099162 | 0.7319939418114051 | 0.5986584841970366 | 0.15601864044243652 |
| Dim_2 | 0.15599452033620265 | 0.05808361216819946 | 0.8661761457749352 | 0.6011150117432088 | 0.7080725777960455 |
| Dim_3 | 0.020584494295802447 | 0.9699098521619943 | 0.8324426408004217 | 0.21233911067827616 | 0.18182496720710062 |

Example

# Feature Extraction!! Word Embeddings

- **Other Word Embeddings (Word2Vec, GloVe):**
  - Dense vectors capturing semantics.
  - CBOW: Predicts center word given context.
  - Skip-Gram: Predicts context given center word.

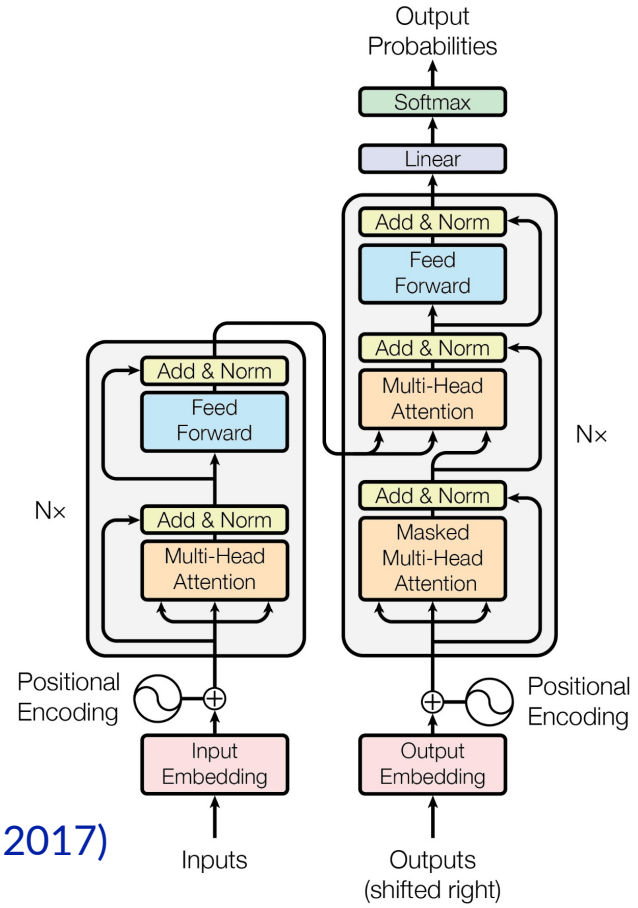
"The cat **sat** on the mat"




CBOW Skip-Grams

# Feature Extraction!! Attention

- Each token embedding “looks at” other tokens.
- Attention assigns weights to decide which tokens are most relevant.
- Output: contextualized embeddings (dynamic, depend on surrounding words).




(Vaswani, 2017)

# Text Analysis

- **Part-of-Speech (POS) Tagging:** Identifies roles of words (noun, verb, adjective).
- **Named Entity Recognition (NER):** Finds entities (people, places, organizations, dates).
- **Dependency Parsing:** Maps out grammatical structure (who did what to whom?).
- **Sentiment Analysis:** Determines emotional tone (positive, negative, neutral).
- **Topic Modeling:** Extracts overarching topics within a text or corpus.
- **Natural Language Understanding (NLU):** Infers deeper meaning and relationships, helping handle synonyms/ambiguities.

# Model Training & Inference

1. **Data Preparation:** Preprocessed text → numeric features (e.g., BoW vectors, embeddings).
2. **Train a Model:**
   a. Classical: Naive Bayes, logistic regression.
   b. Neural Networks: e.g., a feedforward net with an embedding layer, or a Transformer-based model.
3. **Evaluation & Fine-Tuning:**
   a. Use validation data to measure accuracy, F1-score, etc.
   b. Adjust hyperparameters, address errors.
4. **Deployment:** Predict or generate text for new data (e.g., chatbots, translation apps).

# Challenges & Risks in NLP

## Biased Training

- Using data that under-represents certain groups leads to biased outputs.
- Web-scraped data can inadvertently reflect societal biases.

## Misinterpretation

- Speech-to-text errors from dialects, slang, or background noise.
- Garbage in, garbage out (GIGO) problem.

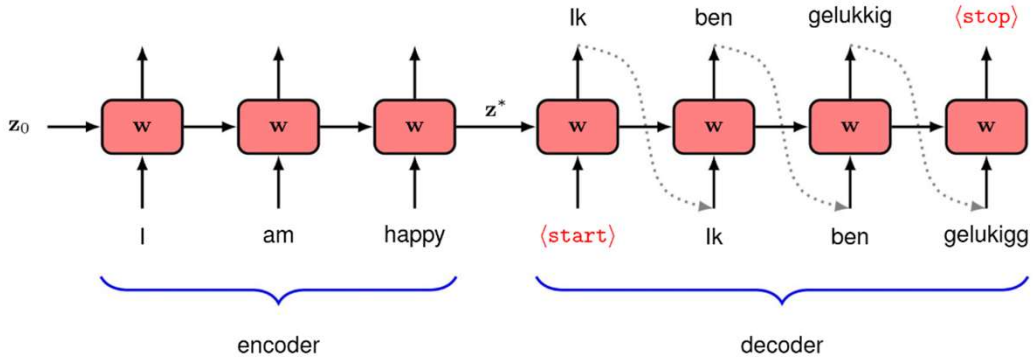
## New Vocabulary

- Language is dynamic; new words or slang appear constantly.
- Models must adapt or risk confusion.

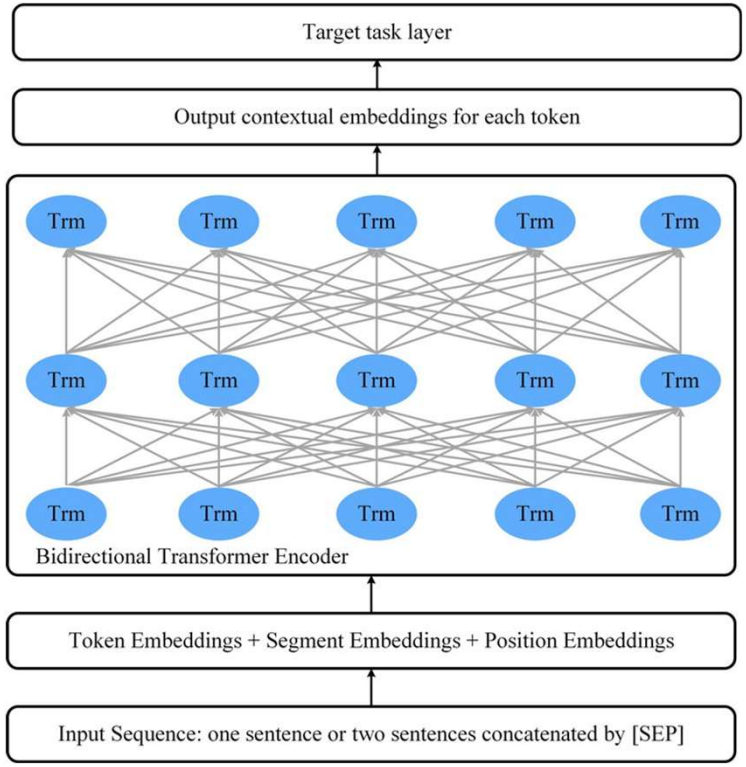
## Tone of Voice / Non-verbal Cues

- Sarcasm or emphasis can change meaning.
- Current text-based NLP struggles to fully capture these nuances

# Putting things together




Example of an RNN for language translation (typically before 2017)
I.e. Google Translate




Example of BERT for search (2018)
I.e. Google Search